# Анализ проекта AirGrap: выявленные проблемы и рекомендации по доработке

## к курсовому проекту «Проектирование и разработка модуля регистрации и обработки программных ошибок»

Студент группы ИСиП/23-1 Цоколов Даниил Дмитриевич

Настоящий документ подготовлен по результатам анализа исходного кода серверной части мессенджера AirGrap (проект AirGrapServer на C#/.NET 10 и SQL-скрипты базы данных PostgreSQL). Цель документа — систематизировать выявленные недостатки модуля регистрации и обработки программных ошибок и смежных компонентов, оценить связанные с ними риски и предложить конкретные меры доработки. Все указания на файлы соответствуют текущей структуре проекта; нумерация пунктов сквозная и используется в последующих таблицах.

Замечания распределены по трём группам по убыванию значимости: критические (безопасность и обработка ошибок), надёжность и качество обработки ошибок, качество кода и именование. Для каждого пункта указаны проблемное место в коде, возможные последствия и рекомендуемое действие. Пункты 1–13 частично использованы в параграфе 3.4 курсового проекта и в презентации к защите; настоящий документ содержит их полный перечень.

# 1 КРИТИЧЕСКИЕ ЗАМЕЧАНИЯ (БЕЗОПАСНОСТЬ И ОБРАБОТКА ОШИБОК)

Таблица 1 – Критические замечания

| № | Проблема | Место в проекте | Риск | Рекомендация |
|---|---|---|---|---|
| 1 | Отсутствует глобальный обработчик исключений | Program.cs | необработанные ошибки возвращают стандартный ответ сервера с потенциальным раскрытием деталей | подключить app.UseExceptionHandler с единым безопасным ответом |
| 2 | HomeController.TestConect не обрабатывает исключения | Controllers/ HomeController.cs | при отказе СУБД возникает необработанное исключение | обернуть вызов в try/catch или опираться на глобальный обработчик |
| 3 | Клиенту возвращается текст исключения (new { error = ex.Message }) | Controllers/ UserController.cs | раскрытие внутренних деталей приложения | возвращать обобщённый текст; детали — только в журнал |
| 4 | Сервис выводит ошибки в консоль (Console.WriteLine) | Servises/ DBServise.cs | часть ошибок не попадает в файловый журнал | заменить на вызов Logger.AddLog |
| 5 | Не журналируются успешные операции и события входа | Servises/ DBServise.cs | отсутствует аудит безопасности (ГОСТ Р 51583-2014) | фиксировать регистрацию и аутентификацию: пользователь, время, результат |
| 6 | Файл исключений аутентификации пуст; в аутентификации перехватывается UserAlreadyExistsException | Exceptions/ AuntificationExceptions.cs, Controllers/UserController.cs | семантически неверная обработка ситуации входа | добавить InvalidCredentialsException и использовать его |

| № | Проблема | Место в проекте | Риск | Рекомендация |
|---|---|---|---|---|
| 7 | Скрипт триггеров ссылается на несуществующие объекты (products, set_updated_at); таблица log пуста | database/ triggers.sql | аудит на уровне СУБД не работает | переписать триггеры для users/sessions/messages, заполняющие log |
| 8 | Пароль подключения по умолчанию хранится в коде | Datasurse/ ConectionString.cs | риск компрометации учётных данных | использовать user-secrets или переменные окружения |

Перечисленные замечания затрагивают контур защиты информации: от них зависит, не раскроет ли приложение внутреннюю структуру системы и сможет ли администратор обнаружить и расследовать инцидент. Рекомендуется выполнить их в первую очередь.

## 2 НАДЁЖНОСТЬ И КАЧЕСТВО ОБРАБОТКИ ОШИБОК

Таблица 2 – Замечания по надёжности

| № | Проблема | Место в проекте | Риск | Рекомендация |
|---|---|---|---|---|
| 9 | Запись журнала без метки времени, уровня, типа исключения и стека | Logs/loger.cs | диагностика ошибок и расследование инцидентов затруднены | расширить формат записи; в перспективе — ILogger или Serilog |
| 10 | Логгер не потокобезопасен (одновременная запись из нескольких запросов) | Logs/loger.cs | нарушение и потеря записей журнала | синхронизировать запись (lock) или перейти на готовую библиотеку |
| 11 | Несогласованность кодов HTTP (NullInputFieldException → 409 Conflict) | Controllers/ UserController.cs | клиент неверно интерпретирует причину ошибки | использовать 400 BadRequest |
| 12 | Токен сессии передаётся в адресе запроса (GET /user/{token}) | Controllers/ UserController.cs | токен попадает в журналы сервера и историю браузера | передавать токен в заголовке Authorization |
| 13 | Отсутствует ограничение частоты попыток аутентификации | Controllers/ UserController.cs | возможен перебор паролей | добавить rate limiting и временную блокировку |

Замечания этой группы не создают прямой угрозы безопасности, однако снижают диагностическую ценность журнала и предсказуемость поведения API. Их устранение существенно упростит сопровождение приложения.

## 3 КАЧЕСТВО КОДА И ИМЕНОВАНИЕ

Таблица 3 – Замечания по качеству кода

| № | Проблема | Место в проекте | Риск | Рекомендация |
|---|---|---|---|---|
| 14 | Опечатки в наименованиях каталогов и файлов (Servises, Datasurse, Auntification, loger, qwery.sql) | структура проекта | снижение читаемости, риск ошибок сопровождения | переименовать: Services, Datasource, Authentication, Logger.cs, query.sql |
| 15 | Опечатки в строках ответов («Server is Actiive», «Internal server error pliase try again») | Controllers/ HomeController. cs, UserController.cs | небрежные сообщения API | исправить тексты сообщений |
| 16 | Параметр метода называется nicname | Servises/ DBServise.cs, Servises/IDBServ ise.cs | несоответствие имени поля DTO | переименовать в nickname |
| 17 | README.md указывает тему «Дипломная работа» | README.md | документация не соответствует проекту | актуализировать описание проекта |
| 18 | Комментарии в .gitignore сохранены в неверной кодировке | .gitignore | нечитаемые комментарии в репозитории | пересохранить файл в UTF-8 |

# 4 ПЕРСПЕКТИВНЫЕ УЛУЧШЕНИЯ

Помимо исправления выявленных недостатков, для развития проекта рекомендуется:

подключить полноценную библиотеку журналирования (Serilog, NLog) со структурным форматом записей, фильтрацией и ротацией файлов;

использовать встроенный интерфейс ILogger ASP.NET Core вместо собственного статического класса;

реализовать защиту от перебора паролей: временную блокировку после серии неудачных попыток входа;

ввести refresh-токены и настраиваемое время жизни сессии вместо фиксированного лимита трёх сессий;

ограничить доступ к документации Scalar/OpenAPI в производственной среде;

добавить принудительное перенаправление на HTTPS (UseHttpsRedirection) для развёртывания вне локальной сети.

## 5 РЕКОМЕНДУЕМЫЙ ПОРЯДОК РАБОТ

Этап 1 (безопасность): пункты 1, 3, 4, 6 — глобальный обработчик ошибок, обобщённые ответы клиенту, журналирование сервиса, исключения аутентификации. Эти изменения локальны и не затрагивают схему базы данных.

Этап 2 (аудит и хранение): пункты 5, 7, 8 — аудит успешных операций, переработка триггеров и таблицы log, вынесение параметров подключения в пользовательские секреты.

Этап 3 (качество): пункты 9–18 и перспективные улучшения — расширение журнала, согласование кодов HTTP, переименования и актуализация документации. Переименования каталогов удобно выполнять средствами рефакторинга среды разработки, чтобы не нарушить ссылки в файлах проекта.

Все предложенные изменения не затрагивают основную функциональность мессенджера и могут выполняться последовательно без остановки разработки.